

GARAD-SLAM: 3D GAussian splatting for Real-time Anti Dynamic SLAM

[†]Mingrui Li¹, [†]Weijian Chen², Na Cheng¹, Jingyuan Xu¹, Dong Li^{3*} and Hongyu Wang^{1*}

Abstract—The 3D Gaussian Splatting (3DGS)-based SLAM system has garnered widespread attention due to its excellent performance in real-time high-fidelity rendering. However, in real-world environments with dynamic objects, existing 3DGS-based SLAM systems often face mapping errors and tracking drift issues. To address these problems, we propose GARAD-SLAM, a real-time 3DGS-based SLAM system tailored for dynamic scenes. In terms of tracking, unlike traditional methods, we directly perform dynamic segmentation on Gaussians and map them back to the front-end to obtain dynamic point labels through a Gaussian pyramid network, achieving precise dynamic removal and robust tracking. For mapping, we impose rendering penalties on dynamically labeled Gaussians, which are updated through the network, to avoid irreversible erroneous removal caused by simple pruning. Our results on real-world datasets demonstrate that our method is competitive in tracking compared to baseline methods, generating fewer artifacts and higher-quality reconstructions in rendering.

I. INTRODUCTION

Currently, the outstanding performance of 3DGS in photo-realistic high-fidelity reconstruction has attracted significant attention, leading to the emergence of 3DGS-based SLAM systems [1]–[5]. For fields like AR/VR and robotics, a SLAM system with real-time high-fidelity dense reconstruction capabilities holds great significance. On the other hand, existing 3DGS-based SLAM systems such as SplatAM [6] and MonoGS [2] exhibit limitations when confronted with real-world environments. Real-world environments often contain a large number of moving objects, and traditional SLAM systems usually employ semantic-based methods to remove these interferences [7]–[14]. However, current 3DGS-based SLAM systems cannot simply apply these feature-based removal or optical flow segmentation methods. These methods can only correct tracking errors but cannot resolve artifacts caused by dynamic noise inputs.

Recently, SLAM systems based on neural implicit representations have made progress in addressing these dynamic interferences, such as DN-SLAM [15], NID-SLAM [16], DDN-SLAM [17], and RoDyn-SLAM [18]. However, neural implicit methods still have some unresolved challenges. For example, DN-SLAM and DDN-SLAM rely on traditional

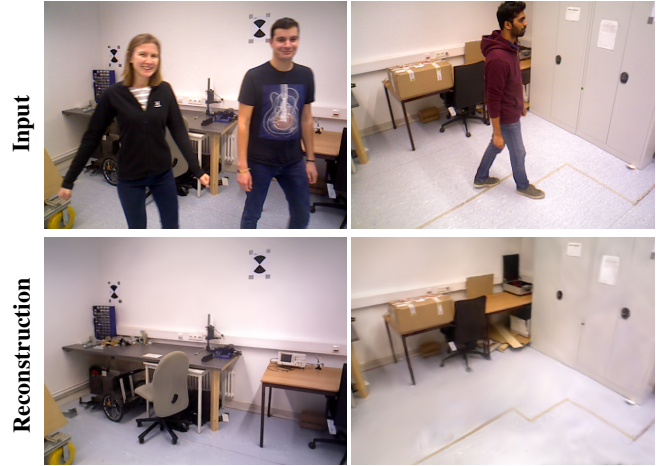


Fig. 1: Rendering result in BONN datasets. The reconstruction result is generated by our GARAD-SLAM.

front-end pose correction, and while DDN-SLAM adds additional rendering penalties during the back-end rendering process, it fails to resolve artifacts from multi-view synthesis fundamentally. NID-SLAM and RoDyn-SLAM use optical flow-based estimation methods for tracking, where RoDyn-SLAM removes sampled rays with dynamic labels while applying warp loss to track camera motion. However, these methods have limitations in real-time performance and scene geometry accuracy, with tracking and mapping remain loosely coupled.

Therefore, we propose GARAD-SLAM, a 3DGS-based SLAM system designed for dynamic environments. Unlike traditional methods, we do not perform semantic segmentation on the feature points in the tracking phase. Instead, we directly build a conditional random field (CRF) [19] based on feature and position for Gaussian rendering in the back-end and perform segmentation. Using our Gaussian pyramid network, we map this segmentation back to the front-end feature points and apply labels. This allows us to remove back-end rendering artifacts while correcting pose errors. To ensure the accuracy of dynamic Gaussian labels, we build a sparse optical flow validation from the front-end feature points to recover incorrectly labeled Gaussians in the back-end. This results in tight integration of tracking and rendering in our system. Finally, we gradually reconstruct the static scene through rendering loss, eliminating the influence of dynamic Gaussians. Our test results on several real-world datasets demonstrate that our method achieves state-of-the-art results in rendering accuracy and runtime performance.

Our contributions can be summarized as follows:

Corresponding authors: Dong Li and Hongyu Wang.

[†]Mingrui Li and [†]Weijian Chen contributed equally.

¹Mingrui Li, Na Cheng, Jingyuan Xu, Hongyu Wang are with the School of Information and Communication Engineering, Dalian University of Technology, Dalian, 116024, China 2905450254@mail.dlut.edu.cn

²Weijian Chen is with the School of Aeronautics and Astronautics, Sun Yat-sen University, Shenzhen, 528406, China.

³Dong Li is with the Faculty of Science and Technology, University of Macau, Macao, 999078, China.

- 1) We introduce GARAD-SLAM, which to the best of our knowledge, is the first 3DGS-based SLAM system designed specifically for dynamic scenes.
- 2) We achieve tight coupling and mutual enhancement between tracking and mapping, in contrast to traditional methods that only focus on tracking correction.
- 3) We propose a dynamic point removal method based on a conditional CRF for Gaussians, and introduce a rendering loss for dynamic Gaussian labels. Additionally, we provide an error label recovery method that utilizes sparse optical flow for feature points corresponding to the Gaussians.
- 4) Our experimental results on real-world datasets demonstrate that our method achieves state-of-the-art performance in terms of tracking and rendering accuracy.

II. RELATED WORK

Neural Radiance Fields (NeRF) [20] and 3D Gaussian Splatting (3DGS) [21] have garnered widespread attention in recent years as effective methods for scene reconstruction [22]–[26]. These approaches have shown exceptional performance, particularly in real-time photorealistic rendering, and their integration with RGB-D SLAM systems has demonstrated impressive results. iMAP [27] was the first SLAM system based on NeRF, but its single MLP model limited its ability to represent complex scenes. However, with the emergence of 3DGS-based SLAM, neural implicit representation faced significant challenges as 3DGS-based SLAM achieved better results in both tracking accuracy and rendering. 3DGS strikes a balance between implicit and explicit representations. For instance, SplatAM [6] uses frame-to-frame tracking and per-pixel output to represent 3DGS scenes, while MonoGS [2] enables pure RGB scene tracking and reconstruction. Nonetheless, these SLAM systems struggle in dynamic real-world scenarios, as dynamic interferences lead to erroneous feature matching or pixel misalignment, resulting in tracking failures or loss. Furthermore, dynamic noise in the input causes rendering artifacts during mapping, posing a significant challenge to current SLAM systems.

As NeRF-SLAM continues to evolve, researchers are increasingly focusing on handling dynamic scenes. DN-SLAM [15] employs optical flow estimation to remove dynamic feature points from the front-end, correcting tracking errors similarly to traditional methods. However, it overlooks the rendering artifacts caused by noisy inputs in the back end. DDN-SLAM [17] combines deep learning-based detection with depth segmentation and introduces additional rendering penalties. RoDyn-SLAM [18] uses optical flow estimation on keyframes to obtain dynamic masks, removing dynamic rays for updates, but its real-time performance and rendering accuracy are limited. NID-SLAM [16] also utilizes optical flow estimation to generate dynamic masks and background completion to mitigate dynamic noise interference, but its tracking performance is constrained. Our approach integrates the strengths of these methods while addressing the issue of decoupled tracking and mapping. We use a Gaussian pyramid

network to apply front-end feature points to Gaussians for densification and directly apply dynamic segmentation for rendering correction on Gaussians, mapping them back to the tracked feature points. This method tightly integrates the system, allowing tracking and mapping to mutually enhance each other, ultimately constructing a static scene map.

III. METHOD

The framework of our system is shown in Fig. 2. In tracking and gaussian mapping threads, we maintain Tagged Gaussians, which present a point cloud P_i that contains ORB-matched feature points p_i , rotation r , density σ , and spherical harmonics descriptors SH . These Tagged Gaussians allow the system to optimize tracking using graph-based optimization, while simultaneously learning the corresponding mapping by backpropagating the loss between the original images and rendering images. In Sec III-A, we proposed a method for determining dynamic labels of Gaussians using CRF, while also employing a Gaussian Pyramid Network to back-project and filter feature points on the front-end. In Sec III-B, we developed a sparse optical flow detection to further filter feature points and recover incorrect Gaussian labels. In Sec III-C, we introduced dynamic rendering penalties, utilizing updates from the Gaussian Pyramid Network to progressively optimize Gaussians with dynamic labels, ultimately achieving static rendering results.

A. Dynamic Gaussian Validation Based on CRF

In traditional semantic SLAM methods, dynamic objects are often handled by removing feature points associated with them in the front-end [11]–[13], [28]–[30]. This helps correct the erroneous feature matches caused by dynamic objects. Subsequently, dynamic objects are removed through pixel-level segmentation, ensuring accurate back-end mapping. However, in SLAM systems based on 3DGS, while pixel-level segmentation can effectively remove dynamic interference, it often incurs significant computational costs and may inadvertently eliminate some static Gaussians, thereby affecting the quality of back-end mapping. Conversely, the absence of pixel-level segmentation may result in residual transient interference from dynamic objects, ultimately leading to rendering artifacts.

To address this issue, we propose a method that integrates a Gaussian pyramid network with CRF segmentation. A Gaussian pyramid is a multi-scale representation generated by applying Gaussian smoothing and downsampling to the original image, capturing varying levels of detail. Unlike traditional approaches, we directly construct a CRF based on the Gaussians in the back-end and eliminate dynamic Gaussians during the pyramid optimization process. Once dynamic Gaussians are identified, we generate a Gaussian mask and map it back to the front-end feature points through the Gaussian pyramid network, correcting pose errors in front-end tracking. This method effectively removes artifact interference from the rendering process.

In particular, the system assumes a static scene for tracking during the first ten frames, generating Tagged Gaussians.

To address this issue, we propose a sparse optical flow-based motion state verification method for dynamic Gaussians. This method maps dynamic Gaussians to the front-end's dynamic points through masks and uses sparse optical flow verification to accurately identify and recover those Gaussians that were incorrectly labeled as dynamic. Sparse optical flow verification reduces computational complexity while fully utilizing optical flow information between keyframes, ensuring that mistakenly deleted static Gaussians can be effectively recovered.

Specifically, we process the feature points in the front-end that correspond to the Tagged Gaussians. Through the 2D-2D matching relationship, we obtain matching point sets $P^i = \{p_k^i | k = 1, 2, \dots, n\}$ and $P^j = \{p_k^j | k = 1, 2, \dots, n\}$ from frames i and j , where n is the number of successfully matched feature points. To construct the Gaussian distribution model, we select a certain number of static points, which are considered relatively stable in the scene, and whose optical flow vectors follow a Gaussian distribution.

For each static matching point (p_k^i, p_k^j) , we use the Lucas-Kanade optical flow method to calculate its optical flow V_k , and then use these optical flow vectors to construct a Gaussian distribution model. The mean $\mu = \frac{1}{n} \sum_{k=1}^n V_k$ and the covariance matrix Σ of the model are computed based on the optical flow vectors of the static points as follows:

$$\Sigma = \frac{1}{n-1} \sum_{k=1}^n (V_k - \mu)(V_k - \mu)^T \quad (8)$$

The chi-square value χ^2 of the vector between two matched points is defined as:

$$\chi^2 = (V_k - \mu)^T \Sigma^{-1} (V_k - \mu) \quad (9)$$

The chi-squared value has 2 degrees of freedom, and a threshold $\chi_{0.05}^2$ is set. If the calculated chi-squared value falls within this threshold, the point is considered to not deviate significantly from the Gaussian distribution of the static points. Therefore, it can be marked as a static point and mapped back to the Tagged Gaussians, where it will be relabeled as a Static Gaussian.

After completing the labeling of static points and the recovery of Gaussians, we proceed with camera pose optimization using graph-based optimization. The Levenberg-Marquardt (LM) algorithm is employed to minimize the error between the matched Static Gaussians P_i^s and feature points p_i , where $i \in \mathcal{X}$ represents the set of matched features. This can be expressed as:

$$\{R, t\} = \arg \min_{R, t} \sum_{i \in \mathcal{X}} \rho \left(\|p_i - \pi(\cdot)(RP_i^s + t)\|_{\Sigma}^2 \right) \quad (10)$$

Where $R \in SO(3)$, $t \in \mathbb{R}^3$, $\rho(\cdot)$ represents a robust kernel function, $\pi(\cdot)$ is the projection function, and Σ is the covariance matrix of the feature point.

C. Dynamic Gaussians Penalty Optimization

Dynamic Gaussians can significantly affect scene rendering accuracy. The typical approach is to directly remove detected dynamic Gaussians, but this method heavily relies on the precise identification of dynamic Gaussians. If they are not accurately recognized, residual dynamic objects may cause artifacts or blurriness. Conversely, mistakenly removing static Gaussians can lead to the loss of key geometric structures, resulting in holes or incompleteness in the reconstructed scene.

During training with the Gaussian pyramid network, we monitor label changes and prune Gaussians with low opacity and large size to enhance efficiency. However, occlusions or changes in appearance can lead to mistakenly pruning important Gaussians, which is irreversible. To prevent this, we temporarily retain dynamically labeled Gaussians and only delete them after prolonged consistent labeling. Additionally, we introduce a dynamic Gaussians penalty constraint during optimization to minimize the impact of truly dynamic Gaussians on rendering while avoiding the mistaken deletion of static Gaussians, thereby preserving both scene quality and critical information.

The optimization function with a penalty term for dynamic Gaussians not only considers the contribution of dynamic Gaussians to rendering errors but also reduces the interference with the static structure of the scene by balancing the credibility of dynamic labels and the observation duration. The specific optimization function can be expressed as:

$$\mathcal{L} = \lambda_{p-ssim} \mathcal{L}_{p-ssim} + \lambda_{dyn} \mathcal{L}_{dyn} \quad (11)$$

Where λ is the weight coefficient and $\mathcal{L}_{dyn} = \sum_{i \in \mathcal{G}_D} \alpha_i^2$.

For $\mathcal{L}_{p-ssim}$, we consider both brightness and SSIM :

$$\mathcal{L}_{p-ssim} = (1-\lambda) |I_r - I_{gt}| + \lambda (1-SSIM(I_r - I_{gt})) \quad (12)$$

Where I_r is the rendered image and I_{gt} is the real image.

During the rendering stage, static Gaussians can be rasterized to synthesize corresponding images with keyframe poses. The rendering process is formulated as:

$$C(R, t) = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (13)$$

Where N is the number of static Gaussians, c_i present the color converted from SH , and α_i is equal to $\sigma \cdot \mathcal{G}$, $\mathcal{G}$ denotes 3D Gaussian splatting algorithm [21].

In the Gaussian pyramid-based network optimization process, the Gaussian parameters are first supervised and trained at the top layer of the pyramid, then progressively optimized down through the subsequent layers. This optimization process can be described as:

$$t_i : \arg \min \mathcal{L}(I_r^i, GP^i(I_{gt})), \quad i = n, \dots, 1, 0 \quad (14)$$

Where $\mathcal{L}(I_r^i, GP^i(I_{gt}))$ is the loss function between the rendered image and the ground truth image. $GP^i(I_{gt})$ represents the ground truth image corresponding to the i -th layer of the Gaussian pyramid.

TABLE I: RESULTS OF METRIC ATE ON TUM DATASET.

The best-performing results are highlighted in bold, while the second-best results are underscored, “X” denotes the tracking failures. The metric unit is [m]

Sequences	ORB-SLAM3		Photo-SLAM		SplaTAM		DDN-SLAM		NID-SLAM		GARAD-SLAM	
	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD
fr3/walking/xyz	0.3623	0.1855	0.2862	0.1591	0.4216	0.2538	0.0140	<u>0.0085</u>	0.0623	0.0372	<u>0.0155</u>	0.0080
fr3/walking/xyz_val	0.6514	0.4354	0.6041	0.4025	0.8798	0.7954	<u>0.0132</u>	<u>0.0063</u>	0.0471	0.0146	0.0129	0.0057
fr3/walking/static	0.1500	0.0850	0.1127	0.0209	0.4612	0.2883	<u>0.0100</u>	<u>0.0052</u>	0.0581	0.0368	0.0061	0.0027
fr3/walking/static_val	0.2856	0.0605	0.2762	0.0615	0.7418	0.4571	<u>0.0113</u>	0.0076	0.0230	0.0089	0.0104	<u>0.0081</u>
fr3/walking/rpy	0.6058	0.2781	0.5782	0.2687	X	X	<u>0.0390</u>	<u>0.0252</u>	0.6080	0.4107	0.0360	0.0224
fr3/walking/rpy_val	0.3331	0.1769	0.3258	0.1698	X	X	<u>0.0291</u>	0.0176	X	X	0.0280	<u>0.0184</u>
fr3/walking/half	0.2291	0.1013	0.2051	0.1201	0.7321	0.6127	<u>0.0230</u>	<u>0.0132</u>	0.6080	0.4107	0.0229	0.0124
fr3/walking/half_val	0.3236	0.1820	0.3458	0.2048	1.1425	1.0410	<u>0.0243</u>	<u>0.0157</u>	0.0821	0.0477	0.0236	0.0139
Avg.	0.3676	0.1881	0.3418	0.1759	0.5474	0.4310	<u>0.0205</u>	<u>0.0124</u>	0.1861	0.1208	0.0194	0.0115

TABLE II: RESULTS OF METRIC ATE ON BONN DATASET.

The best-performing results are highlighted in bold, while the second-best results are underscored, “X” denotes the tracking failures. The metric unit is [m]

Sequences	ORB-SLAM3		Photo-SLAM		SplaTAM		DDN-SLAM		NID-SLAM		GARAD-SLAM	
	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD	ATE	STD
crowd	0.4244	0.3035	0.4568	0.3514	1.945	1.182	0.0180	<u>0.0108</u>	0.1804	0.1021	<u>0.0182</u>	0.0091
crowd2	0.9257	0.5681	0.9425	0.5735	3.582	1.9664	0.0230	0.0114	0.3701	0.2559	<u>0.0244</u>	<u>0.0123</u>
person_tracking	0.6176	0.3291	0.5715	0.2912	0.2598	0.2883	0.0430	<u>0.0171</u>	0.1002	0.0612	<u>0.0462</u>	0.0150
person_tracking2	0.6724	0.3254	0.6518	0.3073	0.5117	0.4571	0.0380	0.0130	0.1473	0.0988	<u>0.0387</u>	<u>0.0133</u>
synchronous	0.7824	0.4365	0.8071	0.4536	X	X	<u>0.0398</u>	<u>0.0267</u>	X	X	0.0248	0.0207
synchronous2	1.3425	0.4865	1.3229	0.4695	X	X	<u>0.0194</u>	<u>0.0128</u>	X	X	0.0069	0.0038
balloon	0.0592	0.0418	0.0340	0.0166	2.4199	1.0138	0.0180	0.0094	0.0355	0.0186	<u>0.0296</u>	<u>0.0119</u>
balloon2	0.2030	0.1377	0.2369	0.0735	1.3569	0.0724	0.0410	0.0267	<u>0.0304</u>	<u>0.0177</u>	0.0254	0.0118
Avg.	0.6284	0.3286	0.6279	0.3171	1.2594	0.6225	<u>0.0300</u>	<u>0.0160</u>	0.1080	0.0693	0.0268	0.0122

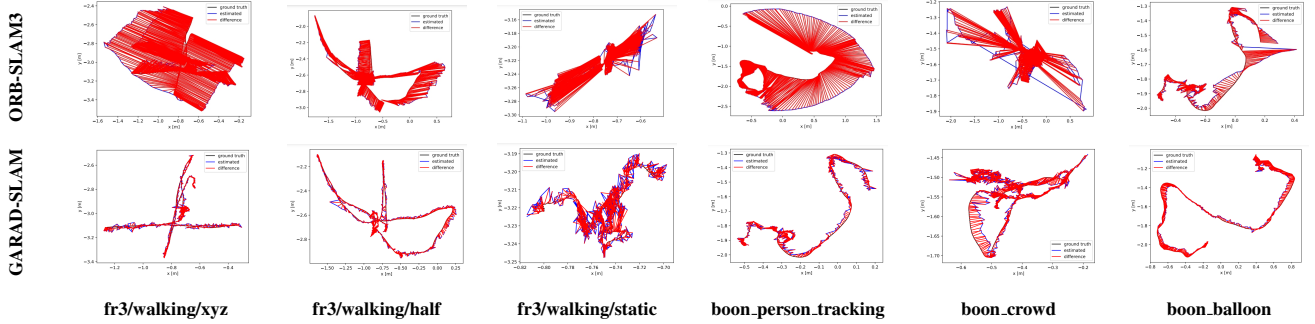


Fig. 3: ATE for ORB-SLAM3 and GARAD-SLAM on some sequences of TUM and BONN datasets.

IV. EXPERIMENTS

A. Implementation and Experiment Setup

Datasets and Metrics. We utilize two dynamic datasets, TUM RGB-D [31] and BONN RGB-D [32], to evaluate the performance of our algorithm. The accuracy metric that we assess is the absolute trajectory error (ATE), which represents the global stability of the trajectory. We use root mean square error (RMSE) and standard deviation (STD) to express the robustness and stability of the system. Since it is not feasible to obtain ground truth reconstructions in dynamic environments, we do not compute SSIM or PSNR. Instead, we present a qualitative comparison of the rendering results.

Implementation details. All our datasets were processed on a computer equipped with an Intel Core i9-14900K CPU and an RTX 4080 Ti GPU. The rendering process utilized the

stochastic gradient descent (SGD) method, with the Gaussian pyramid set to three levels (*i.e.*, $n = 2$). And the loss weight $\lambda_{p-ssim} = 0.8$, $\lambda_{dyn} = 0.2$, $\lambda = 0.2$. We compare our approach with ORB-SLAM3 [33], Photo-SLAM [5], DDN-SLAM [17], NID-SLAM [16], and SplaTAM [6], with NID-SLAM being re-implemented by us.

B. Evaluation of Tracking

Table I presents the results for eight sequences from the TUM RGB-D dataset. Our approach leverages Gaussian segmentation in dynamic environments, coupled with front-end optical flow verification, resulting in advanced tracking performance compared to existing methods. Table II illustrates the performance of our system on the more complex BONN RGB-D dataset, demonstrating its superior efficacy in handling challenging scenarios.

Additionally, we present the ATE visualization results for six sequences from the TUM and BONN datasets. As shown in Fig. 3, our system achieves more accurate localization in dynamic environments.

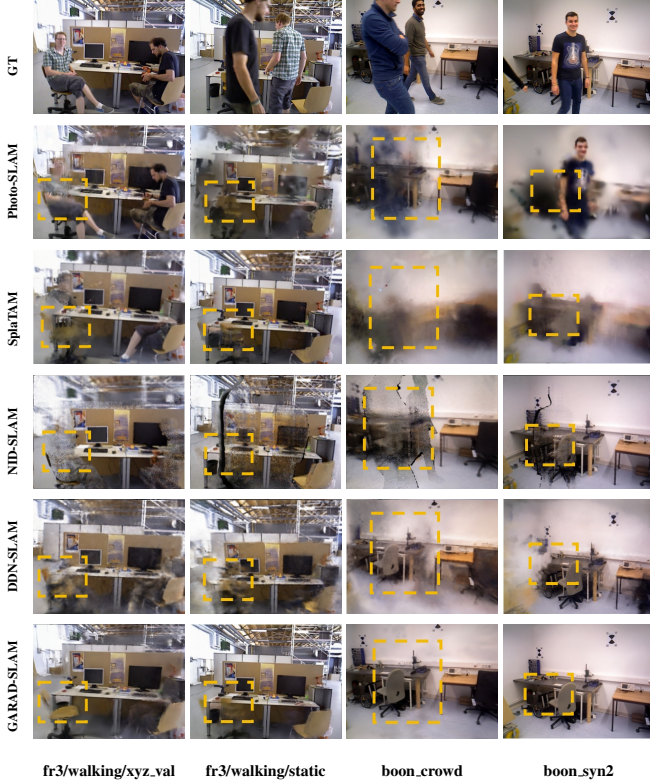


Fig. 4: Gaussian map of TUM RGB-D and BONN datasets.

C. Visualization of Gaussian Map

To demonstrate the performance of dynamic scene reconstruction, we compared the rendered images with the ground truth poses obtained from the generated Gaussian map, using the same viewpoint to compare with other methods. Four challenging sequences were selected: boon_crowd, boon_syn2 from the BONN dataset and f3_walk_xyz_val, f3_walk_static from the TUM RGB-D dataset. As shown in Fig. 1 and Fig. 4, our method demonstrates superior performance in mitigating artifacts in dynamic environments. This highlights the ability of our approach to efficiently segment dynamic Gaussians in real-time, reducing their impact during optimization and achieving improved rendering quality.

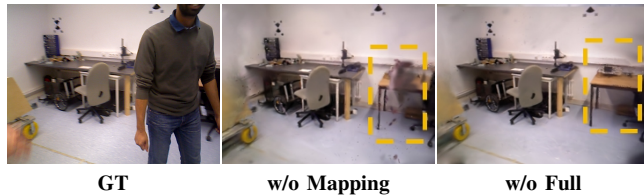


Fig. 5: Comparison of rendering quality.

D. Ablation Study

In this section, we conduct various experiments to verify the effectiveness of our method. w/o CRF refers to the

TABLE III: ABLATION STUDY RESULTS ON TUM AND BONN DATASETS. THE METRIC UNIT IS [m]

Dataset	Metric	w/o CRF	w/o Flow	w/o Mapping	w/o Full
TUM	ATE	0.3418	0.0232	0.0212	0.0194
	STD	0.1759	0.0154	0.0125	0.0114
BONN	ATE	0.6287	0.0306	0.0274	0.0267
	STD	0.3183	0.0156	0.0127	0.0122

TABLE IV: RUNTIME ON FR3/WK/XYZ USING RTX 4080 Ti.

	NID-SLAM	DDN-SLAM	SplatAM	Ours
Tracking FPS	2.7	20.5	52.6	56.4
Mapping FPS	3.1	50.1	45.5	54.5

method without CRF segmentation, while w/o Flow indicates performing only CRF segmentation without optical flow verification. w/o Mapping means we do not use the back-end Gaussian mapping strategy but instead employ a traditional approach that only removes front-end feature points. w/o Full refers to performing CRF segmentation along with sparse optical flow verification.

We compute the average ATE and STD on the TUM and BONN datasets in Tab. I and Tab. II to demonstrate how different methods affect system tracking performance. The results are presented in the Tab. III indicate that using CRF-based Gaussians segmentation combined with front-end optical flow verification to restore static points facilitates better pose estimation.

For rendering quality, we compared the mapping quality of w/o Mapping and w/o Full. As illustrated in Fig. 5, w/o Mapping, which removes feature points only in the front-end, may lead to artifacts in the back-end rendering. And w/o Full directly applies CRF segmentation on Gaussians in the back-end, effectively reducing the artifacts introduced during the rendering process.

E. Time Consumption Analysis

As shown in Tab. IV, we report the time consumption (frames per second, FPS) of tracking and mapping with other SOTA methods. The results indicate that our method is faster than previous methods.

V. CONCLUSIONS

We propose GARAD-SLAM, which effectively addresses the issues of tracking drift and mapping errors in 3DGS-based SLAM systems under dynamic scenes. Our approach introduces an anti-dynamic strategy based on back-end Gaussian mapping and front-end sparse optical flow validation. Through step-by-step updates based on neural networks, we achieve accurate removal of dynamic interference and pose correction. We tightly integrate tracking and mapping processes, which mutually enhance each other. Experimental results demonstrate that our method significantly mitigates the impact of transient interference and artifacts compared to other baseline methods, achieving top performance on real-world datasets. Future research will focus on the adaptation of our method for mobile applications and lightweight deployment.

REFERENCES

- [1] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "Gs-slam: Dense visual slam with 3d gaussian splatting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 595–19 604.
- [2] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 18 039–18 048.
- [3] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-slam: Photo-realistic dense slam with gaussian splatting," *arXiv preprint arXiv:2312.10070*, 2023.
- [4] T. Deng, Y. Chen, L. Zhang, J. Yang, S. Yuan, D. Wang, and W. Chen, "Compact 3d gaussian splatting for dense visual slam," *arXiv preprint arXiv:2403.11247*, 2024.
- [5] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-slam: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 584–21 593.
- [6] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "Splatam: Splat track & map 3d gaussians for dense rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 357–21 366.
- [7] B. Bescos, J. M. F  cil, J. Civera, and J. Neira, "Dynaslam: Tracking, mapping, and inpainting in dynamic scenes," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [8] L. Xiao, J. Wang, X. Qiu, Z. Rong, and X. Zou, "Dynamic-slam: Semantic monocular visual localization and mapping based on deep learning in dynamic environment," *Robotics and Autonomous Systems*, vol. 117, pp. 1–16, 2019.
- [9] J. He, M. Li, Y. Wang, and H. Wang, "Ovd-slam: An online visual slam for dynamic environments," *IEEE Sensors Journal*, vol. 23, no. 12, pp. 13 210–13 219, 2023.
- [10] Y. Liu and J. Miura, "Rds-slam: Real-time dynamic slam using semantic segmentation methods," *IEEE Access*, vol. 9, pp. 23 772–23 785, 2021.
- [11] X. Hu, Y. Zhang, Z. Cao, R. Ma, Y. Wu, Z. Deng, and W. Sun, "Cfp-slam: A real-time visual slam based on coarse-to-fine probability in dynamic environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 4399–4406.
- [12] Z.-J. Du, S.-S. Huang, T.-J. Mu, Q. Zhao, R. R. Martin, and K. Xu, "Accurate dynamic slam using crf-based long-term consistency," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 4, pp. 1745–1757, 2020.
- [13] J. Huang, S. Yang, T.-J. Mu, and S.-M. Hu, "Clustervo: Clustering moving instances and estimating visual odometry for self and surroundings," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2168–2177.
- [14] W. Dai, Y. Zhang, P. Li, Z. Fang, and S. Scherer, "Rgb-d slam in dynamic environments using point correlations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 1, pp. 373–389, 2020.
- [15] C. Ruan, Q. Zang, K. Zhang, and K. Huang, "Dn-slam: A visual slam with orb features and nerf mapping in dynamic environments," *IEEE Sensors Journal*, vol. 24, no. 4, pp. 5279–5287, 2024.
- [16] Z. Xu, J. Niu, Q. Li, T. Ren, and C. Chen, "Nid-slam: Neural implicit representation-based rgb-d slam in dynamic environments," *arXiv preprint arXiv:2401.01189*, 2024.
- [17] M. Li, J. He, G. Jiang, and H. Wang, "Ddn-slam: Real-time dense dynamic neural implicit slam with joint semantic encoding," *arXiv preprint arXiv:2401.01545*, 2024.
- [18] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, "Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields," *IEEE Robotics and Automation Letters*, vol. 9, no. 9, pp. 7509–7516, 2024.
- [19] P. Kr  henb  hl and V. Koltun, "Efficient inference in fully connected crfs with gaussian edge potentials," *Advances in Neural Information Processing Systems*, vol. 24, 2011.
- [20] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [21] B. Kerbl, G. Kopanas, T. Leimk  hler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
- [22] T. Deng, G. Shen, T. Qin, J. Wang, W. Zhao, J. Wang, D. Wang, and W. Chen, "Plgslam: Progressive neural scene representation with local to global bundle adjustment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 657–19 666.
- [23] T. Deng, Y. Wang, H. Xie, H. Wang, J. Wang, D. Wang, and W. Chen, "Neslam: Neural implicit mapping and self-supervised feature tracking with depth completion and denoising," *arXiv preprint arXiv:2403.20034*, 2024.
- [24] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "Nice-slam: Neural implicit scalable encoding for slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 786–12 796.
- [25] M. M. Johari, C. Carta, and F. Fleuret, "Eslam: Efficient dense slam system based on hybrid representation of signed distance fields," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 408–17 419.
- [26] H. Wang, J. Wang, and L. Agapito, "Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 293–13 302.
- [27] E. Sucar, S. Liu, J. Ortiz, and A. J. Davison, "imap: Implicit mapping and positioning in real-time," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6229–6238.
- [28] H. Yin, S. Li, Y. Tao, J. Guo, and B. Huang, "Dynam-slam: An accurate, robust stereo visual-inertial slam method in dynamic environments," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 289–308, 2022.
- [29] Y. Fan, Q. Zhang, Y. Tang, S. Liu, and H. Han, "Blitz-slam: A semantic slam in dynamic environments," *Pattern Recognition*, vol. 121, p. 108225, 2022.
- [30] A. Li, J. Wang, M. Xu, and Z. Chen, "Dp-slam: A visual slam with moving probability towards dynamic environments," *Information Sciences*, vol. 556, pp. 128–142, 2021.
- [31] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.
- [32] E. Palazzolo, J. Behley, P. Lottes, P. Giguere, and C. Stachniss, "Refusion: 3d reconstruction in dynamic environments for rgb-d cameras exploiting residuals," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 7855–7862.
- [33] C. Campos, R. Elvira, J. J. G. Rodr  guez, J. M. Montiel, and J. D. Tard  s, "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.